browser may have a cookie connected to that server. If the server rejects the request, the browser never sends the actual request with the cookie, and the cookie never reaches the server.

But things are different if there is no preflight. If the client has `withCredentials` set to `true`, and there isn't a preflight, the cookie will be sent to the server. This is because the browser has no way of predicting what the value of the Access-Control-Allow-Credentials header will be before sending the actual request. When the browser sees that the Access-Control-Allow-Credentials header isn't set, it will throw an error in the client. But because the client set the `withCredentials` property, the cookie was already sent to the server in the request.

So you can have situations where the server explicitly states that it doesn't want cookies, but the browser sends them anyway. This may sound unintuitive and a bit scary, until you remember that this is already how requests behave, even without CORS. If a client makes a JSON-P request to a server, and the browser has a cookie for that server, the cookie will be included as part of the JSON-P request.

This behavior isn't new to the web. There is a whole class of attacks that can arise from this request-plus-cookie combination called cross-site request forgery (CSRF), and CORS isn't immune to them. Therefore, standard security precautions such as CSRF prevention should be used when making CORS requests. The next chapter will look at what CSRF is and how to prevent it.

USER CREDENTIALS AND ACCESS-CONTROL-ALLOW-ORIGIN

If the Access-Control-Allow-Credentials header is set to `true`, the `*` value can't be used in the Access-Control-Allow-Origin header. Recall that the value `*` in the Access-Control-Allow-Origin header indicates that any origin can make a cross-origin request.

If the Access-Control-Allow-Credentials header is set to `true`, the server must provide an actual origin value in the Access-Control-Allow-Origin header. If the browser sees the header `Access-Control-Allow-Origin: *` used in conjunction with `Access-Control-Allow-Credentials: true`, it will reject the request. Table 5.6 compares the valid and invalid header combinations.

Table 5.6 Valid and invalid combinations for the Access-Control-Allow-Origin and Access-Control-Allow-Credentials headers

Valid	Invalid
`Access-Control-Allow-Origin: http://localhost:1111` `Access-Control-Allow-Credentials: true`	`Access-Control-Allow-Origin: *` `Access-Control-Allow-Credentials: true`

Allowing cookies to be sent on all requests is insecure and can lead to CSRF vulnerabilities. The CORS spec authors wisely chose to avoid the security risk entirely by disallowing this header combination.